THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 04 OF 04

#12 — Ticket Booking System Design

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

04



YOUR STUDY COMPANION

Step-by-step explanation: how to stop double booking

Goal: Design a ticket booking system for concerts, sports events, or movies where multiple customers can attempt to reserve the **same seat at the same time**.

Recommended baseline: Relational database as the **source of truth**, short-lived **seat holds**, **row-level locking or conditional atomic updates**, a reliable **payment state machine**, and **asynchronous notifications**.

How to read this guide: Start with sections 1–7. They explain the whole system with one example. Sections 8–14 add implementation details, failures, scale, and trade-offs. All numbers are **teaching assumptions**, not performance guarantees.

NAVIGATION

Inside this guide

01	1. Understand the problem with one seat
02	2. Why checking availability and then updating is broken
03	3. Decide when a seat becomes unavailable: temporary holds
04	4. High-level system architecture
05	5. The core solution: one database transaction prevents two winners
06	6. Seat holds, expiry, and releasing abandoned inventory
07	7. Payment and booking: why this is the hardest part after locking
08	8. Data model: minimum tables and integrity rules
09	9. API design — what the client actually calls
10	10. Handle very popular events without corrupting inventory
11	11. Failure scenarios and how to respond
12	12. Compare alternative designs — what we reject and when
13	13. Variations: general admission and multi-region booking
14	14. Operational readiness and interview-ready recap
15	Official sources used for implementation choices

CHAPTER

1. Understand the problem with one seat

A concert has **Seat A12**. Alice and Bob both see it as available. They click **Book** almost together.



Correct business outcome: Exactly **one** of them can reserve and eventually own A12. The other must receive **seat no longer available**. A slow page refresh cannot change that rule.

A booking system has two different jobs:

1. **Show availability** quickly to many users. It is acceptable if this display is briefly stale.
2. **Allocate a seat** correctly. This **must not** sell the same seat to two buyers.

Central distinction: `Showing available` is not `owning a seat`. Final ownership must be decided by an authoritative write, not by whatever the UI last displayed.

Example requirements

Requirement	In plain English
Show event and seat map	Browse events and see seats / prices.
Select specific seats	Reserve A12, A13, etc., not just a ticket count.
Hold seats temporarily	Allow customers a short checkout window, e.g., **5 minutes**.
Avoid double booking	At most one active owner per `(event_id, seat_id)`.
Make payment	Payment may be slow, fail, or return an uncertain result.
Confirm booking	Issue a ticket **after** the chosen payment-success conditions are met.
Recover from errors	Expire abandoned holds, retry safely, never create duplicate charges/tickets.
Handle popular events	Keep heavy browsing traffic away from critical seat-locking operations.

Assumptions for our design: Assigned seats; one organizer controls authoritative inventory; the payment provider supports **separate authorization and capture** for the chosen payment method. For general-admission tickets, section 13 explains the different inventory model.

CHAPTER

2. Why checking availability and then updating is broken

A naive API does this:

```
1. SELECT status FROM seats WHERE seat = 'A12';
2. If AVAILABLE, accept booking.
3. UPDATE seats SET status = 'BOOKED' WHERE seat = 'A12';
```

Two requests can both pass step 1 before either reaches step 3:

Moment	Alice's request	Bob's request
1	Reads A12 = AVAILABLE	—
2	—	Reads A12 = AVAILABLE
3	Decides “I can book”	Decides “I can book”
4	Sets A12 = BOOKED	Sets A12 = BOOKED
Problem	**Both could be told booking succeeded**	**Same seat sold twice**

This is a **race condition**: the result depends on the timing of concurrent operations.

Design principle: The availability **check** and seat **reservation change** must be **one protected database operation**, or one short database transaction with locking. A Java `synchronized` block on one application server is not enough because there are many server instances.

CHAPTER

3. Decide when a seat becomes unavailable: temporary holds

We should **not** permanently sell the seat the moment a user clicks it. The user still needs to pay.

Choose a temporary seat hold: The server reserves A12 for Alice for **5 minutes**. While held, Bob cannot reserve it. If Alice abandons checkout, the hold expires and A12 becomes available again.

Seat state progression

```
  AVAILABLE
      |
      | create hold (atomic DB transaction)
      v
    HELD  -------------> AVAILABLE
      |                  hold expired or cancelled
      | begin checkout
      v
PAYMENT_PENDING --------> AVAILABLE
      |                    payment failed / safe expiry
      | payment authorized
      v
BOOKED_PENDING_CAPTURE
      |               |
      | capture paid  | capture fails definitively
      v               v
    BOOKED          AVAILABLE
      |
      | ticket issued through reliable workflow
```

```
      v
CONFIRMED BOOKING
```

`BOOKED` means the seat is permanently allocated in our inventory. `CONFIRMED BOOKING` means payment is complete and ticket fulfillment can be performed. **Capture uncertainty must not automatically release a seat that might already be paid for.**

Alice and Bob example

```
12:00:00  A12 available
12:00:01  Alice holds A12 until 12:05:01  -> success
12:00:02  Bob tries to hold A12          -> conflict, cannot hold
12:03:00  Alice starts protected payment workflow
12:03:20  Payment completes and booking is confirmed
12:03:21  Bob still cannot reserve A12: it is now BOOKED
```

Why holds, not immediate booking? Holds protect the seat during a slow or interrupted checkout without making abandoned selections permanent.

Why not use a 30-minute hold? Long holds make popular events appear sold out even when users never complete payment. Hold duration is a product decision balanced against payment completion time, fairness, and abandonment.

CHAPTER

4. High-level system architecture



```
                        & validation      notifications
```

What each component does — and why

Component	Responsibility	Why we include it
CDN/cache	Serve event descriptions and seat-map snapshots quickly	Popular events generate far more reads than successful purchases.
WAF / waiting room	Filter bots and admit buyers at a controlled rate	Prevent an on-sale spike from overwhelming booking and database capacity.
API Gateway	Authenticate, route, rate-limit	One entry point and protection layer.
Booking Service	Own seat lifecycle and booking rules	Keeps the critical invariant in one domain boundary.
Relational DB	Authoritative seat ownership and transactions	Provides row locks, constraints, and atomic updates.
Payment provider	Authorize, capture, void or refund payments	External financial work cannot be inside the DB transaction.
Outbox + broker	Publish booking events reliably after commits	Email/ticket workers can run asynchronously without losing committed events.
Ticket issuer	Generate unique ticket and entry credential	Tickets should follow confirmed payment, not a speculative UI action.

Important: Redis is **optional**, and useful for caching or a waiting room. It is **not** our final authority for assigning A12. A stale or lost Redis lock must never create a second sold ticket.

CHAPTER

5. The core solution: one database transaction prevents two winners

A relational database can lock the row representing `(concert-7, A12)` while one request changes it.

Decision: pessimistic row-level locking

Pessimistic locking means “assume conflict is possible; lock the seat while deciding.” PostgreSQL provides `SELECT ... FOR UPDATE`, which prevents another transaction from modifying or locking the same row until the first transaction ends.

For a simple **single-seat hold**, an even shorter approach is an **atomic conditional UPDATE**. The database updates only if the seat is available or has an expired ordinary hold:

```
BEGIN;

UPDATE event_seats
SET state = 'HELD',
    hold_id = :new_hold_id,
    hold_expires_at = NOW() + INTERVAL '5 minutes'
WHERE event_id = :event_id
  AND seat_id = :seat_id
  AND (
       state = 'AVAILABLE'
       OR (state = 'HELD' AND hold_expires_at <= NOW())
```

```
  )
RETURNING event_id, seat_id;

-- Success only when exactly ONE row was returned.
-- In the same transaction, insert the hold record and outbox event.
COMMIT;
```

What happens to Alice and Bob? The first `UPDATE` that wins the row lock moves A12 to HELD. The second request waits and then re-checks the row; it no longer satisfies the `WHERE` condition, so it updates **zero** rows. **Zero rows = no seat allocated.**

Important implementation details: Generate a unique hold ID; authenticate the buyer; create the `holds` row in the **same** transaction; if a prior expired hold is reclaimed, mark its record expired consistently. Handle lock timeouts, transaction retries, and expected conflicts separately. The SQL is illustrative, not a complete API implementation.

What if someone books 3 seats together?

A customer wants A12, A13, and A14 **together**. We must either hold **all three** or **none**.

```
BEGIN TRANSACTION
    Lock A12, A13, A14 in a consistent sorted order
    Verify all three are available or safely expired
    If any seat is unavailable: ROLLBACK
    Otherwise: mark all three HELD for one hold_id
               insert hold and outbox record
COMMIT
```

Concept — ACID atomicity: The transaction fully commits or fully rolls back. The customer does not get A12 and A13 if A14 was unavailable. Always acquire seat locks in a consistent order to reduce deadlocks; use a bounded lock wait.

Why not `SKIP LOCKED` for a specific chosen seat? It can hide a temporarily locked seat from the query, producing an inconsistent view. It is useful for queues, but for exact seat selection use short waits, `NOWAIT`, or a clear conflict response instead.

CHAPTER

6. Seat holds, expiry, and releasing abandoned inventory

Store the expiry timestamp in the authoritative database. A hold is not valid just because a background worker has not yet deleted it.

Example row:

```
Event: concert-7   Seat: A12
state: HELD
hold_id: hold-901
hold_expires_at: 12:05:01
```

Two ways to recover expired seats

1. **Lazy expiry (on demand):** When another user tries to hold A12, the booking transaction sees an expired `HELD` row and atomically takes it. This protects correctness even if background jobs are delayed.
2. **Background sweeper:** Periodically finds expired holds, re-checks the current state inside a transaction, and releases still-expired seats. This helps refresh availability displays and cleans old records.

Why both? A cleanup worker can be delayed or fail. Expiry correctness must come from the state and timestamp checked during the authoritative transaction—not from the worker running exactly on time.

Avoid an important expiry race

Once Alice begins payment, we transition her seat from `HELD` to **`PAYMENT_PENDING`** inside a DB transaction and set a **bounded payment deadline**. Ordinary hold reclamation must **never** take `PAYMENT_PENDING` inventory. If payment becomes uncertain, reconciliation decides whether to confirm, void, or release the seat.

Do not blindly release a seat at 5 minutes if payment is already being processed. Otherwise a late payment could succeed after another customer has received the seat.

CHAPTER

7. Payment and booking: why this is the hardest part after locking

We have **two independent systems**:

- Our database decides who owns Seat A12.
- A payment provider decides whether money was authorized or captured.

We cannot wrap both systems in one normal SQL transaction. A network timeout is especially dangerous: **the payment might have succeeded even if our service never received the response**.

Chosen approach: authorize first, finalize allocation, then capture

This design assumes the payment method/provider supports **separate authorization and capture**. Provider-specific constraints must be checked before using this flow.

Step-by-step successful flow:

1. Alice creates a **5-minute hold** on A12 in our database.
2. Alice clicks Pay. In a **short DB transaction**, we verify the hold is hers and not expired, change it to `PAYMENT_PENDING`, and grant a **bounded payment-processing window**. Commit. **Do not hold a SQL row lock while calling the payment provider.**
3. Call the payment provider to **authorize** the charge using a stable payment-operation ID / idempotency key. An authorization reserves funds but is not a final capture.
4. Once authorization is known to have succeeded, use another **short DB transaction** to verify that the seat still belongs to Alice and her payment deadline is valid. Set `BOOKED_PENDING_CAPTURE`, create a booking/payment record, and commit.
5. Call the payment provider to **capture** the authorized amount with an idempotent operation ID. If capture completes, change booking to `CONFIRMED` and seat to `BOOKED` in the DB. Write a `BookingConfirmed` outbox event **in the same transaction**.
6. The outbox relay publishes the event. Ticket issuance and notifications happen asynchronously and idempotently.



```
    |                  |<------ success --------|
    |                  |-- confirm + outbox     |
    |<-- confirmed ----|                        |
    |                  |-- async issue ticket   |
```

Why separate authorization and capture? If the seat allocation fails after authorization, we can usually **void/cancel the authorization** instead of needing to refund an already captured payment. This choice is not available for every payment method.

Payment failure and compensation rules

Situation	Correct next step
Authorization definitively declined	Mark payment failed; release the seat atomically, if no other protected process owns it.
Authorization succeeded but hold lost / deadline expired	Do **not** confirm the booking; void the authorization and reconcile the provider result.
Capture definitively failed	Void remaining authorization if applicable; cancel pending booking and release the seat in a controlled transaction.
Payment / capture timed out; final outcome unknown	Keep seat in protected pending state, query the provider/webhooks, and reconcile **before** releasing or selling it.
Capture succeeded, DB confirmation failed	Retry DB confirmation idempotently; keep the seat protected. Do **not** create another payment.
DB confirmed but email delivery failed	Retry asynchronous event delivery; booking and payment remain confirmed.

Critical rule: Never treat an **HTTP timeout** as proof of **payment failure**. Never sell a seat that is in an unresolved paid-or-unpaid state. Use bounded reconciliation and escalation so seats do not remain stuck indefinitely.

Concept — Saga: This is a distributed business workflow with steps and **compensations**. A failure cannot roll back an external payment call with SQL `ROLLBACK`. We must perform a compensating action, such as voiding an authorization or issuing a refund where necessary.

CHAPTER

8. Data model: minimum tables and integrity rules

Keep the first version small. Use one authoritative seat row, separate history records, and a durable outbox.

Table	Important columns	Purpose
`events`	`event_id`, `name`, `starts_at`, `status`	Event information.
`event_seats`	`(event_id, seat_id)` **PK**, `state`, `hold_id`, `hold_expires_at`, `booking_id`	**Current owner and state of each seat. Source of truth.**
`holds`	`hold_id` **PK**, `customer_id`, `status`, `expires_at`, `created_at`	Who started a temporary reservation.
`hold_seats`	`hold_id`, `event_id`, `seat_id`	Supports multi-seat reservations.
`bookings`	`booking_id` **PK**, `customer_id`, `status`, `amount`, `currency`	Booking lifecycle and audit history.
`payments`	`payment_id`, `booking_id/hold_id`, `provider_ref`, `status`, `operation_key` **UNIQUE**	Authorization, capture, reconciliation.

`tickets`	`ticket_id` **PK**, `booking_id`, `event_id`, `seat_id`, `status`	Issued ticket records.
`outbox`	`event_id` **PK**, `aggregate_id`, `event_type`, `payload`, `published_at`	Reliable event publication after commit.

The most important invariant

> For each `(event_id, seat_id)`, there is **at most one current active reservation or booking**.

We enforce that using the **single authoritative row** in `event_seats`, updated transactionally. The composite primary key prevents two different inventory rows for the same seat.

Ticket records may need a **unique partial index on active tickets** so a cancelled historical ticket does not block a legitimate future reissue/resale. Exact ticket transfer and resale rules depend on the business.

```
CREATE TABLE event_seats (
  event_id       BIGINT NOT NULL,
  seat_id        TEXT NOT NULL,
  state          TEXT NOT NULL CHECK (
                    state IN ('AVAILABLE','HELD','PAYMENT_PENDING',
                              'BOOKED_PENDING_CAPTURE','BOOKED')),
  hold_id        UUID NULL,
  hold_expires_at TIMESTAMPTZ NULL,
  booking_id     UUID NULL,
  PRIMARY KEY (event_id, seat_id)
);
```

Note: This is a teaching schema. Real production constraints would check valid combinations of state and ownership, and the payment-processing deadline would be stored explicitly. A separate booking state machine tracks capture and refund/void activity.

CHAPTER

9. API design — what the client actually calls

Request	What it does	Important response
`GET /events/{id}/seats`	Reads a cached snapshot of seats and prices	Fast, may be slightly stale.
`POST /holds`	Atomically holds selected seat IDs	`201` + hold ID and expiry; or `409` if a seat is unavailable.
`GET /holds/{id}`	Returns authoritative hold state	Available, held, pending, expired, etc.
`POST /holds/{id}/checkout`	Starts an idempotent payment workflow	Booking/payment operation ID; may return pending.
`GET /bookings/{id}`	Returns final or pending booking state	Clients should poll or subscribe without resubmitting payment.
`POST /webhooks/payment-provider`	Processes verified provider events	Idempotent event processing; not a public client endpoint.

Why idempotency is mandatory

Alice presses **Pay**. The request times out. She presses Pay again. Without safeguards, two payment requests might be created.

Choose an `Idempotency-Key` per checkout operation and store the unique operation record in the database. Repeating the same operation should return the existing booking/payment state rather than creating another one. Pass a stable key to the payment provider too, **following that provider's actual idempotency contract**.

Idempotency ≠ locking:

- **Locking** prevents **two different customers** from owning A12.
- **Idempotency** prevents **one customer's retry** from creating a duplicate payment or booking.

We need **both**.

CHAPTER

10. Handle very popular events without corrupting inventory

A large event might have **50,000 seats** and **hundreds of thousands of interested users**. Most users are browsing and refreshing. Only a small portion are successfully booking.

Separate reads from writes

```
Seat-map requests (very many) -> CDN / cache / read replicas
                                        |
                              cached availability

Hold and checkout requests -> Booking Service -> Primary DB
                                        |
                                authoritative lock
```

Why? The seat map can tolerate minor staleness; the allocation cannot. Even when a cached map says A12 is free, the final `POST /holds` must re-check in the authoritative database.

Use a waiting room for extreme on-sale spikes

```
100,000 interested customers
            |
         Waiting room
        /          \
   admitted users   queued users
        |
     Booking API
        |
     Inventory DB
```

The waiting room caps how many buyers reach the booking service at once. It improves stability, but **does not replace DB correctness**. Fairness and bot resistance are separate product/security concerns.

Simple illustrative capacity math

Assume **60,000 active browsers**, each refreshing the seat map every **5 seconds**. That creates about **12,000 seat-map reads/second** (`60,000 / 5`). We should serve most of these from caches rather than one primary database.

Suppose only **1,000 hold attempts/second** reach the booking service. Our critical write sizing and lock-contention analysis starts there. Both numbers are **examples, not predictions**; benchmark actual distributions, including everyone trying to grab the same premium seat.

Scaling progression: Begin with well-indexed SQL transactions. Add caching and admission control. Then partition inventory by event/section if measured pressure justifies it—while maintaining atomic multi-seat allocation within one transactional boundary. Sharding different requested seats across independent databases complicates all-or-none bookings.

CHAPTER

11. Failure scenarios and how to respond

What fails?	What could go wrong?	Required behavior
Two users choose A12	Overselling	One DB transaction wins; the other gets conflict/no seat.
App crashes after creating a hold	Seat stays unavailable	Hold TTL + lazy expiry + cleanup worker.
Worker runs late	Stale expired holds persist	Checkout/hold transaction checks expiry itself.
Payment succeeds after expiry	Charged user without seat	Verify ownership/deadline before confirmation; void/refund late payment.
Payment result unknown	Duplicate capture or early release	Query/reconcile provider outcome using stable operation ID; keep seat protected.
Client retries checkout	Duplicate booking or charge	Durable idempotency record at API and provider.
DB transaction deadlocks	Slow or failed attempt	Lock seats in stable order, bound wait, retry safe DB transactions.
DB primary fails	Lost/uncertain writes	Recovery plan with tested replication, durable commits and reconciliation. Do not promise zero data loss without an architecture supporting it.
Event broker fails	Ticket email never sent	Transactional outbox, retry relay, idempotent consumers.
Cached seat map is stale	User clicks already held seat	Authoritative hold returns `409`; refresh affected seats.
Payment captured but confirmation write fails	Paid but booking uncertain	Retry confirmation against protected seat; reconcile before release.

Transactional outbox explained simply

A booking is confirmed in the database, but the service crashes **before** publishing `BookingConfirmed` to the broker. Without an outbox, ticket issuance and notifications may never happen.

Fix: In the **same DB transaction** that confirms the booking, insert a row in `outbox`. An independent relay later sends it to the broker. The relay may publish more than once, so consumers must deduplicate by event/ticket ID.

```
ONE DB TRANSACTION
   Update booking = CONFIRMED
   Update seat = BOOKED
```

```
    Insert outbox(BookingConfirmed)
COMMIT
          |
        Outbox relay (retryable)
          |
        Message broker
          |
      Ticket issuer / email (idempotent)
```

CHAPTER

12. Compare alternative designs — what we reject and when

Option	Why consider it?	Why not default for this design?
SQL row lock / conditional UPDATE	Correctness, easy transaction boundary	**Chosen.** Contention on one hot seat is unavoidable, but guarantees are clear.
Optimistic locking with version	Good when conflicts are rare; `UPDATE ... WHERE version = x`	For heavily contested seats, many retries and poor customer experience. Still valid if measured contention is low.
Redis `SET NX` hold lock	Extremely fast, TTL built in	Lock expiry/failover alone cannot prove durable ownership; still need DB transaction and fencing/validation.
Distributed lock service	Cross-process coordination	Adds complexity; a correctly modeled database row can already arbitrate ownership.
Queue all hold requests	Smooth bursts and preserve an ordering policy	Added latency; still need authoritative allocation and retries. Best as admission control for extreme demand.
Event-driven eventual booking only	Great for notifications and workflows	**Not** appropriate for two customers claiming one specific seat without a synchronous authoritative conflict decision.
Multi-region active-active writes	Local low-latency booking globally	Conflicting concurrent seat sales require coordination; choose an event/home-region inventory writer for safety.

Pessimistic vs. optimistic locking in one sentence

- **Pessimistic:** lock first, then decide; good under high conflict.
- **Optimistic:** read a version, then commit only if unchanged; good when conflicts are uncommon.

Both rely on an **atomic database write** to enforce the final ownership invariant.

CHAPTER

13. Variations: general admission and multi-region booking

A. General admission (no assigned seats)

Instead of a unique A12, an event has **500 tickets remaining**. The invariant becomes:

> Confirmed tickets + valid holds must never exceed capacity.

Use an **atomic inventory decrement / reservation** (`UPDATE inventory SET available = available - :qty WHERE available >= :qty RETURNING ...`) in a transaction. Store hold quantity and restore it on safe cancellation/expiry. Beware a single hot inventory row for very popular events; introduce partitioned allocations or bounded quota leases only when necessary, with exact aggregate-capacity accounting.

B. Multi-region deployment

If a buyer in Europe and a buyer in the US both request **A12**, independently writable regional databases might each accept the booking. **Asynchronous replication does not prevent double selling.**

Safer baseline: Pick an **authoritative inventory region per event**. Reads can be regional and cached, but **all seat-hold writes for that event** go to its designated writer. After a regional failure, only promote a replacement if inventory state is recovered consistently and old writers are fenced off. This adds cross-region latency, but preserves one authority for a seat.

C. Reservations across several rows/sections

If a booking spans many seats, lock them in deterministic order and commit them as a single transaction. If those seats live in **different databases**, the all-or-none guarantee becomes significantly harder. Prefer inventory partitioning that keeps a normal group booking in one transaction boundary, or explicitly redesign the requirement and compensation logic.

CHAPTER

14. Operational readiness and interview-ready recap

What to measure

- **Hold success/conflict rate** per event and section; popular seats have legitimate contention.
- **Seat-lock wait time, DB deadlocks, connection pool usage**; watch hot-seat pressure.
- **Expired holds and their release delay**; too many stale holds reduce sellable capacity.
- **Payment pending / uncertain counts and age**; no unresolved payment should be ignored.
- **Captures without confirmed bookings, bookings without captures**; reconcile automatically and alert.
- **Outbox backlog and ticket issuance delay**; booking confirmation and ticket delivery are separate.
- **Overbooking invariant violations**; treat any such event as a critical incident.

The key decisions in one table

Interview question	Answer	Reason
How do we stop double booking?	Atomic SQL update or row lock	One seat, one authoritative winner.
Why temporary holds?	Give user a bounded checkout window	Avoid permanently locking inventory during abandoned checkout.
Why SQL over only Redis?	Durable ownership and ACID transactions	A lost/expired cache lock must not sell the seat twice.
How to book 3 seats together?	One transaction; lock rows in sorted order	All-or-none inventory allocation.

What about payment failure?	Saga-like state machine + compensation	Payment provider and booking DB are different systems.
What about timeout?	Idempotency + reconciliation	A timeout is not proof of payment failure.
How do we scale reads?	CDN/cache/read model	Browsing can be eventually consistent.
How do we scale peak writes?	Admission control + benchmark + partition carefully	Avoid overwhelming the authoritative DB.
Why outbox?	Reliable async publication after booking commit	No lost confirmation event during service crash.
Multi-region?	One authoritative writer per event	Prevent independent regions selling the same seat.

A strong two-minute design explanation

> “The key invariant is that one seat can have only one active reservation or confirmed owner. I would keep seat inventory in a relational database and use a short transaction with row locking or a conditional update to atomically create a temporary hold. A five-minute hold allows checkout without making abandoned seats permanently unavailable. For multi-seat requests, all rows must be allocated in one transaction or none are allocated. The public seat map can come from a cache, but every hold must be checked against the authoritative database. For payment I would move the hold to a protected payment-pending state, authorize through the provider using an idempotency key, confirm allocation, then capture and issue the ticket when payment is known to have completed. I would not keep database locks open over network calls. If payment times out, I would reconcile before releasing the seat. A transactional outbox makes confirmed booking events reliable. For flash sales, I would add a waiting room, bot protection and cached reads. Across regions, I would assign each event one authoritative inventory writer to avoid double selling.”

CHAPTER

Official sources used for implementation choices

1. PostgreSQL: Explicit Locking — `SELECT FOR UPDATE`, row locks, deadlock considerations.
2. PostgreSQL: SELECT locking clause — `NOWAIT`, `SKIP LOCKED`, and locking behavior.
3. PostgreSQL: Constraints — primary keys, unique constraints, and partial unique indexes.
4. Stripe: Place a hold on a payment method — separate authorization and capture; support varies by payment method.
5. Stripe: Idempotent requests — safe retries and idempotency keys.
6. AWS Prescriptive Guidance: Transactional outbox pattern — publish events reliably after database commits.

Note: PostgreSQL, Stripe, and AWS are **reference implementations**, not required products or claims about any particular employer's stack. The core design patterns apply to equivalent technologies.